

Лабораторная работа №4

Цель работы: «Реализация графического пользовательского интерфейса для загрузки изображения в веб-приложение и отправка его на сервер»

Ход работы: в ходе выполнения данной работы необходимо реализовать front-end часть приложения, которое позволит загружать цифровые изображения и отправлять их на сервер для последующей обработки моделью искусственной нейронной сети.

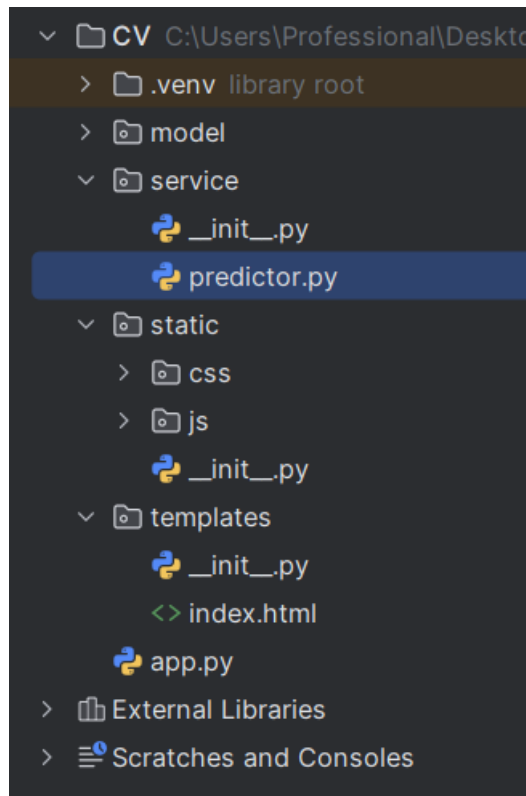


Рисунок 1. – Структура проекта

Реализация цели работы включает в себя модификацию шаблона «index.html», путем добавления соответствующей HTML разметки, добавления CSS стилей и JavaScript кода.

Добавьте в корневую директорию проекта пакет «static», и создайте в нем папки “css” и “js” в которых будут находиться файлы со стилями и JavaScript кодом соответственно. Также создайте пакет “service”, как показано на рисунке 1. В нем необходимо создать файл “predictor.py” в котором будет находиться код обработки полученного цифрового изображения для последующей подачи в нейросетевую модель.

```
<!DOCTYPE html>
<html lang=»ru»>
<head>
  <meta charset=»UTF-8»>
  <meta name=»viewport» content=»width=device-width, initial-scale=1.0»>
  <title>Классификатор геометрических фигур</title>
```

```
<link rel=>stylesheet href=>{{ url_for('static',
filename='css/style.css') }}>
<link
href=>https://fonts.googleapis.com/css2?family=Inter:wght@400;500;600;700&dis
play=swap rel=>stylesheet>
</head>
<body>
<div class=>container>
<header class=>header>
<h1>❖ Классификатор геометрических фигур</h1>
<p>Загрузите изображение квадрата, круга или треугольника для
определения типа фигуры</p>
</header>

<main>
<div class=>upload-section>
<form id=>uploadForm class=>upload-form
enctype=>multipart/form-data>
<div class=>upload-area id=>uploadArea>
<div class=>upload-content>
<svg class=>upload-icon width=>48 height=>48
viewBox=>0 0 24 24 fill=>none stroke=>currentColor stroke-width=>2>
<path d=>M21 15v4a2 2 0 0 1-2 2H5a2 2 0 0 1-
2-2v-4 />

<polyline points=>17 8 12 3 7 8 />
<line x1=>12 y1=>3 x2=>12 y2=>15 />
</svg>
<h3>Перетащите изображение сюда</h3>
<p>или</p>
<button type=>button class=>btn btn-primary
id=>browseBtn>Выберите файл</button>
<input type=>file id=>fileInput name=>file
accept=>image/* hidden>
<p class=>file-info>Поддерживаются: JPG, PNG,
JPEG (макс. 10MB)</p>
</div>
<div class="upload-preview" id="uploadPreview"
style="display: none;">
<img id="previewImage" src="" alt="Preview">
<button type="button" class="btn btn-remove"
id="removeImage">✕</button>
</div>
</div>

<div class="submit-section">
<button type="submit" class="btn btn-submit"
id="submitBtn" disabled>
<span class="btn-text">Распознать фигуру</span>
<span class="btn-loader" style="display:
none;">⌛</span>
</button>
</div>
</form>

<div class="result-section" id="resultSection"
style="display: none;">
<h2>Результат:</h2>
<div class="result-card">
<div class="result-icon" id="resultIcon">🔍</div>
<div class="result-info">
<span class="result-label" id="resultLabel">-
<span class="result-confidence"

```

```

id="resultConfidence">-</span>
    </div>
  </div>
</div>

  <div class="error-section" id="errorSection" style="display:
none;">
    <div class="error-message">
      <span class="error-icon">⚠</span>
      <span id="errorText">Произошла ошибка</span>
    </div>
  </div>
</div>

<div class="info-section">
  <h3>Поддерживаемые фигуры:</h3>
  <div class="shapes-grid">
    <div class="shape-card">
      <div class="shape-icon square">■</div>
      <span>Квадрат</span>
    </div>
    <div class="shape-card">
      <div class="shape-icon circle">○</div>
      <span>Круг</span>
    </div>
    <div class="shape-card">
      <div class="shape-icon triangle">▲</div>
      <span>Треугольник</span>
    </div>
  </div>
</div>
</main>
</div>

<script src="{{ url_for('static', filename='js/script.js') }}"></script>
</body>
</html>

```

Рисунок 2. – Программный код реализации шаблона

Скопируйте код, приведенный на рисунке №2 и замените им код, имеющийся в шаблоне "index.html". Данный код реализует HTML разметку для возможности загрузки изображения, показа пользователю загруженное изображение, а также реализует кнопку для отправки изображений на сервер.

Для стилизации сайта в ранее созданной папке «CSS» создайте файл с названием «style.css», в нем будут находиться стили. В качестве стилей можно использовать код, приведенный на рисунке 3, или реализовать свой код, согласно персональным предпочтениям.

```

* {
  margin: 0;
  padding: 0;
  box-sizing: border-box;
}

body {
  font-family: 'Inter', sans-serif;
  background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
  min-height: 100vh;
  color: #333;
}

```

```
}

.container {
  max-width: 1200px;
  margin: 0 auto;
  padding: 2rem;
}

/* Header */
.header {
  text-align: center;
  color: white;
  margin-bottom: 3rem;
}

.header h1 {
  font-size: 2.5rem;
  font-weight: 700;
  margin-bottom: 0.5rem;
  text-shadow: 2px 2px 4px rgba(0,0,0,0.2);
}

.header p {
  font-size: 1.1rem;
  opacity: 0.95;
}

/* Upload Section */
.upload-section {
  background: white;
  border-radius: 20px;
  padding: 2rem;
  box-shadow: 0 10px 40px rgba(0,0,0,0.1);
  margin-bottom: 2rem;
}

.upload-form {
  display: flex;
  flex-direction: column;
  gap: 1.5rem;
}

.upload-area {
  border: 2px dashed #cbd5e0;
  border-radius: 16px;
  padding: 2rem;
  text-align: center;
  transition: all 0.3s ease;
  position: relative;
  min-height: 300px;
  display: flex;
  align-items: center;
  justify-content: center;
}

.upload-area.dragover {
  border-color: #667eea;
  background: #f7fafc;
}

.upload-content {
  transition: opacity 0.3s ease;
}
```

```
.upload-icon {
  color: #667eea;
  margin-bottom: 1rem;
}

.upload-area h3 {
  font-size: 1.25rem;
  font-weight: 600;
  margin-bottom: 0.5rem;
  color: #2d3748;
}

.upload-area p {
  color: #718096;
  margin-bottom: 1rem;
}

.btn {
  padding: 0.75rem 1.5rem;
  border: none;
  border-radius: 8px;
  font-size: 1rem;
  font-weight: 500;
  cursor: pointer;
  transition: all 0.3s ease;
}

.btn-primary {
  background: #667eea;
  color: white;
}

.btn-primary:hover {
  background: #5a67d8;
  transform: translateY(-2px);
  box-shadow: 0 5px 15px rgba(102, 126, 234, 0.4);
}

.file-info {
  font-size: 0.875rem;
  color: #a0aec0;
  margin-top: 1rem;
}

/* Preview */
.upload-preview {
  position: absolute;
  top: 0;
  left: 0;
  width: 100%;
  height: 100%;
  border-radius: 16px;
  overflow: hidden;
}

.upload-preview img {
  width: 100%;
  height: 100%;
  object-fit: contain;
  background: #f7fafc;
}

.btn-remove {
  position: absolute;
```

```

    top: 10px;
    right: 10px;
    width: 40px;
    height: 40px;
    border-radius: 50%;
    background: rgba(255, 255, 255, 0.9);
    color: #e53e3e;
    font-size: 1.5rem;
    display: flex;
    align-items: center;
    justify-content: center;
    padding: 0;
    box-shadow: 0 2px 10px rgba(0,0,0,0.1);
}

.btn-remove:hover {
    background: white;
    color: #c53030;
}

/* Submit Button */
.submit-section {
    display: flex;
    justify-content: center;
}

.btn-submit {
    background: #48bb78;
    color: white;
    padding: 1rem 3rem;
    font-size: 1.125rem;
    font-weight: 600;
    opacity: 0.6;
    cursor: not-allowed;
}

.btn-submit.active {
    opacity: 1;
    cursor: pointer;
}

.btn-submit.active:hover {
    background: #38a169;
    transform: translateY(-2px);
    box-shadow: 0 5px 20px rgba(72, 187, 120, 0.4);
}

.btn-loader {
    margin-left: 0.5rem;
    animation: spin 1s linear infinite;
}

@keyframes spin {
    0% { transform: rotate(0deg); }
    100% { transform: rotate(360deg); }
}

/* Result Section */
.result-section {
    margin-top: 2rem;
}

.result-section h2 {
    font-size: 1.25rem;
}

```

```
    font-weight: 600;
    margin-bottom: 1rem;
    color: #2d3748;
}

.result-card {
    background: #f7fafc;
    border-radius: 12px;
    padding: 1.5rem;
    display: flex;
    align-items: center;
    gap: 1.5rem;
    border: 2px solid #e2e8f0;
}

.result-icon {
    width: 60px;
    height: 60px;
    background: white;
    border-radius: 50%;
    display: flex;
    align-items: center;
    justify-content: center;
    font-size: 2rem;
    box-shadow: 0 2px 10px rgba(0,0,0,0.1);
}

.result-info {
    flex: 1;
}

.result-label {
    display: block;
    font-size: 1.5rem;
    font-weight: 700;
    color: #2d3748;
    margin-bottom: 0.25rem;
}

.result-confidence {
    display: block;
    font-size: 1rem;
    color: #718096;
}

/* Error Section */
.error-section {
    margin-top: 1.5rem;
}

.error-message {
    background: #fed7d7;
    border-left: 4px solid #e53e3e;
    padding: 1rem;
    border-radius: 8px;
    display: flex;
    align-items: center;
    gap: 0.75rem;
}

.error-icon {
    font-size: 1.25rem;
}
```

```
.error-text {
  color: #c53030;
  font-weight: 500;
}

/* Info Section */
.info-section {
  background: white;
  border-radius: 16px;
  padding: 2rem;
  box-shadow: 0 5px 20px rgba(0,0,0,0.05);
}

.info-section h3 {
  font-size: 1.125rem;
  font-weight: 600;
  color: #2d3748;
  margin-bottom: 1.5rem;
  text-align: center;
}

.shapes-grid {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  gap: 1.5rem;
}

.shape-card {
  text-align: center;
  padding: 1rem;
  background: #f7fafc;
  border-radius: 12px;
  transition: transform 0.3s ease;
}

.shape-card:hover {
  transform: translateY(-5px);
}

.shape-icon {
  font-size: 3rem;
  margin-bottom: 0.5rem;
}

.square {
  color: #4299e1;
}

.circle {
  color: #48bb78;
}

.triangle {
  color: #ed8936;
}

.shape-card span {
  font-size: 0.875rem;
  font-weight: 500;
  color: #4a5568;
}

/* Responsive */
@media (max-width: 768px) {
```



```

.container {
  padding: 1rem;
}

.header h1 {
  font-size: 2rem;
}

.shapes-grid {
  gap: 0.5rem;
}

.result-card {
  flex-direction: column;
  text-align: center;
}

```

Рисунок 3. – CSS стили

Для того чтобы обеспечить реализацию пользовательского интерфейса необходимо подключить обработчики событий на кнопки и другие компоненты. Это достигается за счет использования JavaScript кода. Одна из возможных реализаций приведена на рисунке 4.

Для подключения JavaScript в папке «static» создайте в файл с названием «script.js». В нем будет находиться необходимый JavaScript код. Пример реализации кода приведен на рисунке 4. Можете использовать свою реализацию.

```

document.addEventListener('DOMContentLoaded', function() {
  const uploadArea = document.getElementById('uploadArea');
  const fileInput = document.getElementById('fileInput');
  const browseBtn = document.getElementById('browseBtn');
  const uploadPreview = document.getElementById('uploadPreview');
  const previewImage = document.getElementById('previewImage');
  const removeImageBtn = document.getElementById('removeImage');
  const submitBtn = document.getElementById('submitBtn');
  const uploadForm = document.getElementById('uploadForm');
  const resultSection = document.getElementById('resultSection');
  const errorSection = document.getElementById('errorSection');
  const resultLabel = document.getElementById('resultLabel');
  const resultConfidence = document.getElementById('resultConfidence');
  const resultIcon = document.getElementById('resultIcon');
  const errorText = document.getElementById('errorText');

  let selectedFile = null;

  // Icons for different figures
  const shapeIcons = {
    'square': '■',
    'circle': '○',
    'triangle': '▲'
  };

  // Names of the figures
  const shapeNames = {
    'square': 'Square',
    'circle': 'Circle',
    'triangle': 'Triangle'
  };

```

```
// Drag & Drop handlers
uploadArea.addEventListener('dragover', (e) => {
    e.preventDefault();
    uploadArea.classList.add('dragover');
});

uploadArea.addEventListener('dragleave', (e) => {
    e.preventDefault();
    uploadArea.classList.remove('dragover');
});

uploadArea.addEventListener('drop', (e) => {
    e.preventDefault();
    uploadArea.classList.remove('dragover');

    const files = e.dataTransfer.files;
    if (files.length > 0) {
        handleFileSelect(files[0]);
    }
});

// Click the button to choose a file to upload
browseBtn.addEventListener('click', () => {
    fileInput.click();
});

// Выбор файла через диалог
fileInput.addEventListener('change', (e) => {
    if (e.target.files.length > 0) {
        handleFileSelect(e.target.files[0]);
    }
});

// Удаление выбранного изображения
removeImageBtn.addEventListener('click', () => {
    resetUpload();
});

// Обработка выбранного файла
function handleFileSelect(file) {
    // Проверка типа файла
    if (!file.type.match('image.*')) {
        showError('Пожалуйста, выберите изображение');
        return;
    }

    // Проверка размера файла (макс 10MB)
    if (file.size > 10 * 1024 * 1024) {
        showError('Размер файла не должен превышать 10MB');
        return;
    }

    selectedFile = file;

    // Показ превью
    const reader = new FileReader();
    reader.onload = (e) => {
        previewImage.src = e.target.result;
        uploadPreview.style.display = 'block';
        document.querySelector('.upload-content').style.opacity = '0.3';
        submitBtn.classList.add('active');
        submitBtn.disabled = false;
    };
}
```

```

        // Скрываем предыдущие результаты
        resultSection.style.display = 'none';
        errorSection.style.display = 'none';
    };
    reader.readAsDataURL(file);
}

// Сброс загрузки
function resetUpload() {
    selectedFile = null;
    fileInput.value = '';
    uploadPreview.style.display = 'none';
    previewImage.src = '';
    document.querySelector('.upload-content').style.opacity = '1';
    submitBtn.classList.remove('active');
    submitBtn.disabled = true;
    resultSection.style.display = 'none';
    errorSection.style.display = 'none';
}

// Отправка формы
uploadForm.addEventListener('submit', async (e) => {
    e.preventDefault();

    if (!selectedFile) {
        showError('Пожалуйста, выберите изображение');
        return;
    }

    // Показываем загрузку
    const submitBtnText = document.querySelector('.btn-text');
    const loader = document.querySelector('.btn-loader');
    submitBtnText.style.display = 'none';
    loader.style.display = 'inline-block';
    submitBtn.disabled = true;

    // Скрываем предыдущие результаты
    resultSection.style.display = 'none';
    errorSection.style.display = 'none';

    // Создаем FormData
    const formData = new FormData();
    formData.append('file', selectedFile);

    try {
        const response = await fetch('/predict', {
            method: 'POST',
            body: formData
        });

        const data = await response.json();

        if (response.ok) {
            // Показываем результат
            displayResult(data);
        } else {
            showError(data.error || 'Произошла ошибка при обработке изображения');
        }
    } catch (error) {
        console.error('Error:', error);
        showError('Ошибка соединения с сервером');
    } finally {
        // Скрываем загрузку

```

```

        submitBtnText.style.display = 'inline-block';
        loader.style.display = 'none';
        submitBtn.disabled = false;
    }
});

// Отображение результата
function displayResult(data) {
    const shape = data.class; // ожидаем 'square', 'circle' или
    'triangle'
    const confidence = (data.confidence * 100).toFixed(1);

    resultLabel.textContent = shapeNames[shape] || shape;
    resultConfidence.textContent = `Уверенность: ${confidence}%`;
    resultIcon.textContent = shapeIcons[shape] || '🔍';

    resultSection.style.display = 'block';
    errorSection.style.display = 'none';
}

// Показ ошибки
function showError(message) {
    errorText.textContent = message;
    errorSection.style.display = 'block';
    resultSection.style.display = 'none';
}
});

```

Рисунок 4. – Реализация JavaScript кода.

После запуска приложения с помощью команды

flask run

сервер должен вернуть шаблон с необходимой разметкой и стилями, а также возможностью загружать изображение. Попробуйте загрузить цифровое изображения для тестирования работы приложения, для этого нажмите соответствующую кнопку с названием «Выберите файл» и загрузите файл с изображением.

После чего окно браузера должно выглядеть как показано на рисунке 5.

В дальнейшем, уже на сервере необходимо будет преобразовать полученное цифровое изображение в numpy вектор требуемой формы, поскольку модель искусственной нейронной сети принимает на вход данные именно в таком формате.

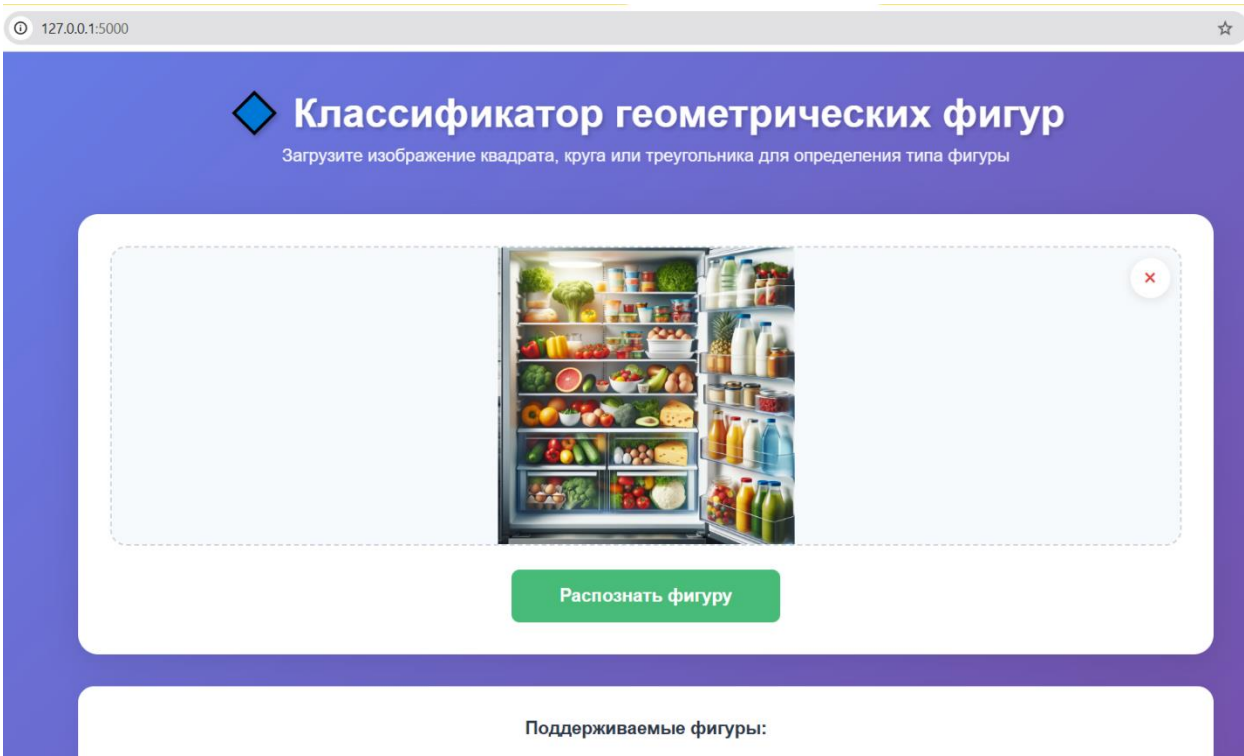


Рисунок 5. – Реализация front-end части приложения

Вопросы:

1. Что такое RESTfull архитектура?
2. Как реализовать обработку событий в JavaScript?
3. Что такое контроллер во фреймворке Flask и какое его назначение в приложении?